SCHOLARMIND

AI Academic Intelligence Agent

HACKATHON CONCEPT BRIEF
A domain-specific AI agent for university students in Malawi.

Analyse past papers. Explain difficult questions. Teach beyond the lecture notes. Help students understand what they know—and what they still need to learn.

Initial target users

University students in Malawi, starting with a focused pilot for selected Computer Science and Information Technology courses at the Malawi University of Business and Applied Sciences (MUBAS).

Core proposition

ScholarMind is not simply an answer-generating chatbot. It uses course materials, examination history, tools and student learning records to support more effective studying.

Prepared as a project-planning document for hackathon discussion

1. Executive overview

ScholarMind is a domain-specific AI academic intelligence agent designed to help university students prepare for examinations and understand challenging course content. Students can upload past examination papers and lecture materials, ask questions, receive structured explanations, and practise concepts through an interactive lecturer-style experience.

The concept combines three core capabilities—Examination Intelligence, Intelligent Question Solving and Second-Level Lecturing—with supporting capabilities for learning-gap detection and study planning.

CORE DIFFERENTIATOR
ScholarMind should not stop after giving an answer. It should explain the reasoning, check whether the student understands, identify weak areas and recommend a useful next learning activity.

The problem

Students often study from disconnected lecture notes, past papers and online resources. They may receive answers without understanding the reasoning, struggle to identify which topics need more attention, or lack access to individualised academic support outside scheduled lectures.

The proposed solution

A single academic agent that can interpret a student's course materials, analyse examination patterns, explain questions step by step and adapt teaching to demonstrated understanding.

Initial scope

- Start with one institution and a small number of courses.
- Support PDF past papers and text-based lecture notes first.
- Build a terminal-based prototype for the hackathon.
- Use a model API rather than training a model from scratch.
- Ground answers in supplied materials and label uncertainty clearly.

2. Core capabilities

A. Examination Intelligence Engine

Students upload past papers, initially as PDFs. ScholarMind extracts questions, organises them by topic and compares papers across years.

• Extract and structure questions from uploaded papers.

• Classify questions by course topic or learning outcome.

• Identify recurring topics and common question formats across uploaded papers.

• Summarise marks allocation and command words such as define, compare, explain and evaluate.

• Generate practice questions based on topics and formats found.

• Produce a preparation summary showing what has appeared in the supplied papers.

Example: A topic appears in three of four uploaded papers. The agent reports this observed frequency and identifies the source papers. It must not claim the topic will appear in the next examination.

B. Intelligent Academic Question Solver

Students type a question, paste text from a paper or ask about uploaded lecture materials. The agent provides a learning-oriented answer rather than only returning a final response.

• Restate what the question is asking.

• Identify relevant concepts and supporting material.

• Explain the solution in logical steps.

• Provide a structured examination-style response where appropriate.

• Explain common mistakes and why they are incorrect.

• Offer a similar practice question to check understanding.

Unique interaction — Answer Deconstruction: show how to construct a strong answer, not merely what the answer is. Suggested mark allocations must be labelled estimates unless an official marking guide was supplied.

C. Second-Level Lecturer / Deep Learning Mode

This mode goes beyond repeating lecture notes. It acts like a patient university-level tutor who explains concepts at different depths, connects theory to practical examples and asks follow-up questions.

• Explain difficult concepts using simpler language and analogies.

• Connect theory to practical or local examples.

• Compare approaches, methods or technologies.

• Ask short questions to check understanding.

• Respond to misunderstandings with a different explanation.

• Offer advanced explanations once the student demonstrates the basics.

Unique interaction — Adaptive Lecturing: use the student's responses to decide whether to reinforce a concept or move to a deeper level. Do not claim mastery based on a single answer.

3. Supporting capabilities

D. Academic Knowledge-Gap Detection

Maintain a simple record of practice attempts and identify areas where the student repeatedly struggles. For the MVP, use quiz results, practice answers and student feedback.

• Track topics practised and question outcomes.

• Highlight topics that need revision.

• Show evidence for a suggested weak area, such as repeated errors on a topic.

• Recommend a relevant lecture section, explanation or practice question.

E. Intelligent Study Planner

Create a revision schedule using the exam date, available study time, course topics and current learning priorities. A simple rule-based planner is sufficient for the first version; AI can explain and adjust the plan.

• Create a day-by-day revision plan.

• Allocate more time to topics needing attention.

• Include past-paper practice and review sessions.

• Adjust the plan when sessions are missed or availability changes.

How the capabilities work together

Stage	Agent activity	Example
1. Input	Receives a question or academic document.	Student uploads past papers and lecture notes.
2. Understand	Identifies course, topic and requested task.	Recognises a database normalisation question.
3. Retrieve	Finds relevant passages in supplied materials.	Retrieves database notes and related questions.
4. Act	Calls the appropriate tool or workflow.	Counts topic frequency or creates a practice quiz.
5. Teach	Generates a grounded explanation and checks understanding.	Explains normalisation, then asks a follow-up.
6. Remember	Stores useful learning progress and preferences.	Records need for more practice with 3NF.

The five capabilities should share one student/course context and a common academic knowledge base. They should be workflows within one agent, not five unrelated chatbots.

4. Proposed technical architecture

A Python terminal application is sufficient for the hackathon. A command-line interface demonstrates agent reasoning, tool use, document analysis and memory without spending time building a full web or mobile interface.

Layer	Suggested technology	Purpose
Application language	Python	Main application and workflow logic.
Agent framework	Pydantic AI or LangGraph	Orchestrate model calls, tools and state. Choose one for the MVP.
Model access	Groq API or OpenRouter	Access supported Qwen, Gemma or Nemotron models.
Document extraction	PyMuPDF	Extract text from text-based PDFs.
OCR (optional)	Tesseract	Extract text from scanned pages; validate OCR output.
Retrieval	ChromaDB (optional)	Find relevant chunks from notes and papers using semantic similarity.
Memory / storage	SQLite	Store profile, document metadata, practice history and progress.
Terminal UI	Rich	Readable terminal screens, tables and prompts.
Configuration	python-dotenv	Load API keys and settings from environment variables.

Understanding the model ecosystem

Name	What it is	Role in the project
Qwen / Gemma / Nemotron	Open-weight model families.	The model generates and reasons over responses.
Hugging Face	Model and dataset platform.	Discover model files, documentation, datasets and hosting options.
Groq	Inference provider.	Runs supported models through an API; check current availability and limits.
OpenRouter	Model API gateway.	Common API interface to models from multiple providers.
Vercel AI SDK	TypeScript/JavaScript AI SDK.	Useful for a JS/TS layer; Python-only projects should use a Python-native SDK or compatible HTTP API.

Recommendation: Start with one gateway—Groq or OpenRouter—and one model that supports the needed context length and tool calling. Verify the exact model identifier, free-tier limits, latency and terms during the hackathon. Not every model is available through every provider.

5. Academic knowledge, memory and skills

Retrieval-Augmented Generation (RAG)

RAG lets the agent answer using relevant passages from documents supplied by the student or project team. It grounds explanations in the correct course materials and reduces unsupported claims.

Step	What happens
1. Upload	Student provides a PDF past paper or lecture note.
2. Extract	Extract text with PyMuPDF; use OCR for scanned documents when needed.
3. Chunk	Split content into sections while preserving page numbers and document identity.
4. Index	Create embeddings and store searchable chunks, or begin with simpler keyword search.
5. Retrieve	Find passages relevant to the student's question.
6. Answer	Send question and retrieved passages to the model; cite document and page where possible.

Memory design

Memory type	MVP approach
Conversation context	Keep recent turns for the current study session.
Student profile	Store course, preferred explanation style and study goals in SQLite.
Learning progress	Store practice attempts, topic tags, scores and feedback.
Document knowledge	Store document metadata and searchable chunks; do not rely on chat history as the document store.

Custom skill: University Academic Mentor — Malawi Edition

A skill is a reusable set of instructions and resources that guides the agent through a specialised task. Create a project skill using the Agent Skills conventions and test it with representative academic questions.

• Teach at university level while adapting explanations to demonstrated understanding.

• Use uploaded course materials as the primary reference when available.

• Show reasoning steps and distinguish explanation from a final exam-style answer.

• Never invent past-paper statistics, official marking schemes, citations or course requirements.

• Use Malawian examples when relevant to the subject.

• Ask diagnostic questions when they help assess understanding.

• State when source material is missing, unclear or contradictory.

Possible file: `skills/university-academic-mentor/SKILL.md`. Skills guide behaviour; tools and retrieval provide the ability to read documents, search information and save progress.

6. Achievable hackathon MVP

The full concept is larger than a short hackathon. Demonstrate a small number of reliable end-to-end workflows rather than five incomplete features.

Priority	Build	Definition of done
Must have	Past-paper analysis	Upload a text PDF, extract questions, tag topics and report observed recurrence across supplied papers.
Must have	Grounded question solver	Ask a question and receive a structured answer based on retrieved course material, with source/page references where possible.
Must have	Deep lecturer mode	Ask for a concept explanation, receive a clear lesson and a follow-up comprehension question.
Should have	Basic memory	Persist course context and a small number of practice results across sessions.
Later	Gap detection and planner	Build after the core workflows are stable and tested.

Suggested terminal experience

```
SCHOLARMIND — Academic Agent
1. Analyse a past paper
2. Ask / solve a question
3. Teach me a topic
4. Review my learning progress
5. Exit

You > Teach me database normalisation at university level.
ScholarMind > Let's build the idea from the beginning...
```

Suggested live demonstration

- **Prepare:** use a small set of permission-cleared past papers and lecture notes from one course.
- **Analyse:** upload two or more papers and show extracted questions and topic counts.
- **Ask:** choose one question and show a step-by-step, source-grounded answer.
- **Teach:** ask the agent to explain a difficult concept, then answer its comprehension question.
- **Remember:** demonstrate recall of the course or a previous practice weakness.

7. Risks, safeguards and quality checks

Risk	Why it matters	Mitigation
Hallucinated answers	Incorrect explanations can mislead students.	Retrieve course sources, show citations/pages, express uncertainty and encourage verification.
False exam predictions	Repeated past topics do not guarantee future exam content.	Report observed historical patterns only; never claim to predict the next paper.
Poor PDF extraction	Scans, tables and mathematical notation may extract incorrectly.	Start with text PDFs, show extracted text for review and add OCR as a fallback.
Invented marking schemes	Suggested marks may be mistaken for official grading.	Label estimates and use official schemes only when supplied and verified.
Privacy and document rights	Papers and student records may contain restricted or personal information.	Use authorised materials, minimise personal data, protect API keys and allow record removal.
API or network failure	Hosted models may be unavailable or rate-limited.	Keep workflows modular; prepare a fallback provider or recorded demo data.

Testing checklist

• Can the system extract questions from the sample PDFs accurately?

• Does topic classification make sense when checked by a student or lecturer?

• Are recurrence counts traceable to the supplied papers and years?

• Does the solver use relevant course material rather than inventing facts?

• Does lecturer mode explain the concept at an appropriate level?

• Does the agent ask a useful follow-up and respond to the student's answer?

• Does memory persist only the intended information?

• Can the app handle missing files, invalid PDFs, API errors and rate limits?

Evaluate correctness, usefulness, source-grounding, response time and whether students understand more after using the agent—not merely whether responses sound fluent.

8. Suggested build sequence

Step	Work	Deliverable
1. Narrow scope	Choose one course, collect a few authorised past papers and notes, define 3–5 test questions.	Small verified test dataset.
2. Prove model access	Connect Python to one model through Groq or OpenRouter; test responses and tool calling.	Working model connection.
3. Document pipeline	Extract PDF text, preserve page references and split content into chunks.	Searchable academic documents.
4. Core workflows	Build paper analysis, grounded question answering and lecturer mode.	Three end-to-end workflows.
5. Tools and memory	Add topic counting, practice-question generation and SQLite learning history.	Agent can act and retain progress.
6. Test and demo	Run test questions, inspect errors and prepare a reliable terminal demo.	Working prototype and demo script.

Suggested team responsibilities

• **Agent and model integration:** prompts, tool calling, provider integration and error handling.

• **Document and retrieval pipeline:** PDF extraction, chunking, indexing and source references.

• **Academic workflows and skill:** exam analysis, answer structure, lecturer behaviour and evaluation examples.

• **CLI, memory and integration:** terminal experience, SQLite persistence, configuration and end-to-end testing.

These are responsibilities, not mandatory separate team members. Combine roles based on team size.

Decisions to make early

• Which first course and academic level will the prototype support?

• Which past papers and lecture materials can the team use with permission?

• Will the prototype use Groq or OpenRouter as its first model gateway?

• Which single agent framework will the team use?

• What must the live demo prove within the available hackathon time?

9. Project pitch

Short pitch

ScholarMind is an AI academic intelligence agent built for university students in Malawi. It analyses past examination papers to reveal observed topic patterns, answers academic questions using students' course materials, and provides adaptive lecturer-style explanations that test understanding. Instead of simply giving students answers, ScholarMind helps them understand concepts, recognise learning gaps and prepare more strategically for examinations.

One-sentence value proposition

From past papers to personal understanding: an AI agent that helps university students study smarter, not just find answers.

Recommended positioning

Position ScholarMind as an academic learning and examination-preparation assistant—not a replacement for lecturers, official course materials or university assessment. Its value is connecting resources, explanations, practice and feedback in one coherent learning workflow.

Useful references

• Groq Console and API: https://console.groq.com/

• OpenRouter model catalogue: https://openrouter.ai/models

• Hugging Face models: https://huggingface.co/models

• Google Gemma: https://ai.google.dev/gemma

• Pydantic AI: https://ai.pydantic.dev/

• LangGraph: https://www.langchain.com/langgraph

• PyMuPDF: https://pymupdf.readthedocs.io/

• ChromaDB: https://docs.trychroma.com/

• Agent Skills directory: https://skills.sh/

Planning note: This document describes a proposed concept and achievable MVP. Verify model names, provider availability, pricing, free-tier quotas and SDK capabilities against current provider documentation before implementation.